- *Exception filters*—Exception filters catch exceptions that occur in the filter pipeline and handle them appropriately. You can use exception filters to write custom, MVC-specific error-handling code, which can be useful in some situations. For example, you could catch exceptions in API actions and format them differently from exceptions in your Razor Pages.
- *Result filters*—Result filters run before and after an action method's `IActionResult` is executed. You can use result filters to control the execution of the result or even to short-circuit the execution of the result.

Exactly which filter you pick to implement will depend on the functionality you're trying to introduce. Want to short-circuit a request as early as possible? Resource filters are a good fit. Need access to the action method parameters? Use an action filter.

Think of the filter pipeline as a small middleware pipeline that lives by itself in the MVC framework. Alternatively, you could think of filters as hooks into the MVC action invocation process that let you run code at a particular point in a request's life cycle.

NOTE The design of the MVC filter pipeline is quite different from the minimal API endpoint filter pipeline you saw in chapter 5. The endpoint filter pipeline is linear and doesn't have multiple types of filters.

This section described how the filter pipeline works for MVC and Web API controllers; Razor Pages use an almost-identical filter pipeline.

21.2 The Razor Pages filter pipeline